M. Moiz Ansari
231-0523
Sec-F

Class Activity- Pipelined RISC-V

X **Q1:** The pipelined RISC-V processor is running the following code snippet. Which registers are being written and which are being read on the fifth cycle? Recall that the pipelined RISC-V processor has a Hazard Unit. You may assume a memory system that returns the result within one cycle.

```
addi s1, zero, 11          # s1 = 11
lw s2, 25(s0)              # s2 = memory[s0+25]
add s3, s3, s4             # s3 = s3 + s4
or s4, s1, s2              # s4 = s1 | s2
lw s5, 16(s2)              # s5 = memory[s2+16]
```

Cycle	Fetch	Decode	Execute	Memory	Writeback
1	addi s1				
2	lw s2 ...	addi s1			
3	add s3, ...	lw s2 ...	addi s1		
4	or s4, ...	add s3, ...	lw s2 ...	addi s1	
5	lw s5 ...	or s4, ...	add s3, ...	lw s2 ...	addi s1
6		lw s5 ...	or s4, ...	add s3, ...	lw s2 ...
7			lw s5 ...	or s4, ...	add s3, ...
8				lw s5 ...	or s4, ...
9					lw s5 ...

Registers Values:

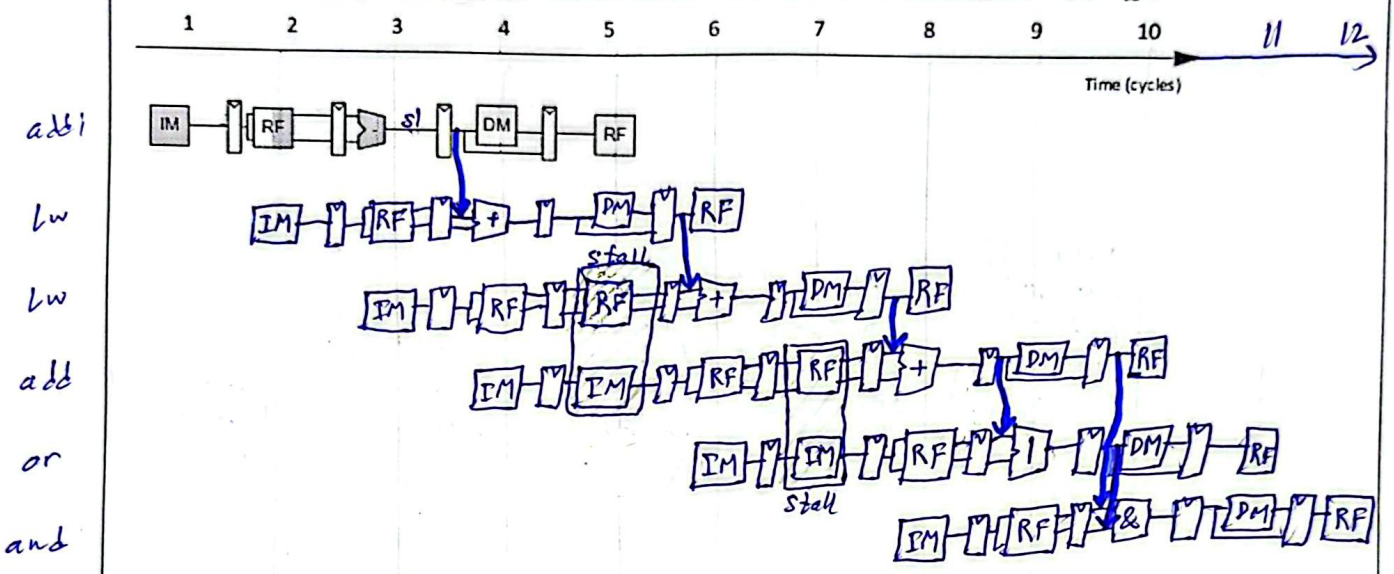
On 5th cycle,
Registers written → s1
Registers read → s1, s2

✓ **Q2:** Using a diagram similar to Figure 7.57, show the forwarding and stalls needed to execute the instructions given below on the pipelined RISC-V processor.

```
addi s1, zero, 11          # s1 = 11
lw s2, 25(s1)              # s2 = memory[36]
lw s5, 16(s2)              # s5 = memory[s2+16]
add s3, s2, s5             # s3 = s2 + s5
or s4, s3, t4              # s4 = s3 | t4
and s2, s3, s4             # s2 = s3 & s4
```

Class Activity- Pipelined RISC-V

Use following diagram notation to draw solution of Q2



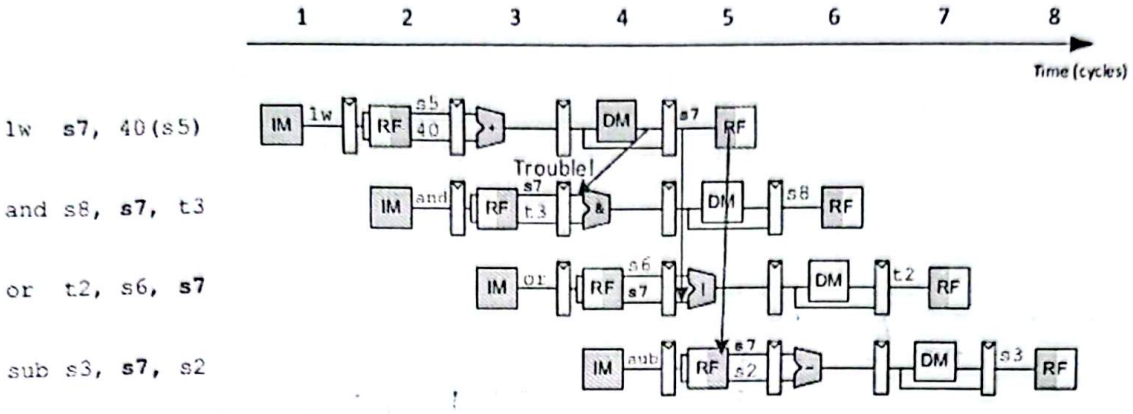


Figure 7.56 Abstract pipeline diagram illustrating trouble forwarding from lw

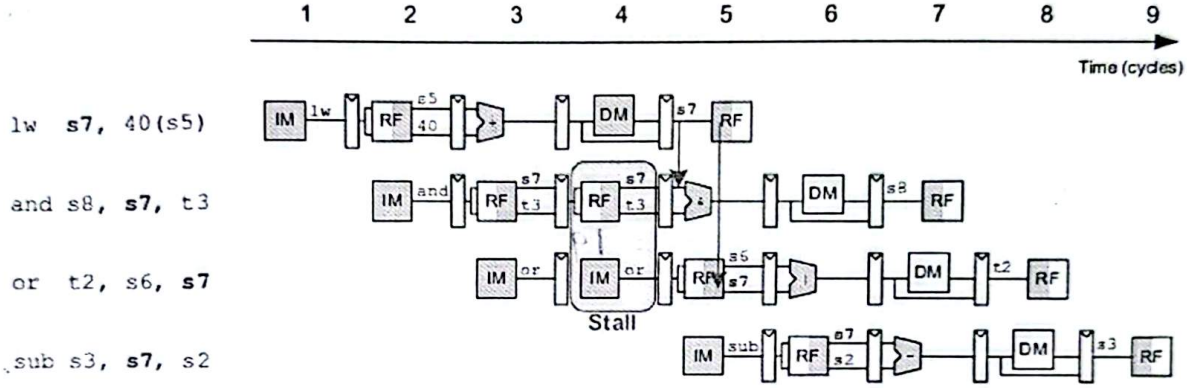


Figure 7.57 Abstract pipeline diagram illustrating stall to solve hazards

Q3: Consider the following instruction sequence in **RISC-V assembly language:**

```
L1:   lw x18, 0(x5)              # x18 = s2, x5 = t0
      sub x18, x18, x19          # x19 = s3
      add x5, x18, x19
      and x5, x5, x6             # x6 = t1
      addi x7, x5, -1            # x7 = t2
      beq x7, x0, L1
      add x28, x5, x18           # x28 = t3
```

Using a diagr
Q5 on the pip

Class Activity- Pipelined RISC-V

A. Show the timing diagram of the above instruction sequence for a **5-stage RISC-V pipeline (IF, ID, EX, MEM, WB)**: without any forwarding but assuming a register read and a write in the same clock cycle "forwards" through the register file.

How many cycles does this code sequence take to execute one iteration?

Instruction	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6	CC 7	CC 8	CC 9	CC 10	CC 11	CC 12	CC 13	CC 14	CC 15/16	CC 17	CC 18	CC 19
lw x18, 0(x5)	IF	ID	EX	MEM	WB													
sub		IF	-	-	ID	EX	MEM	WB										
add					IF	-	-	ID	EX	MEM	WB							
and								IF	-	-	ID	EX	MEM	WB				
addi											IF	-	-	ID	EX/MEM	WB		
beq														IF	=	ID	EX	MEM
add																IF	ID	EX

Cycles for one iteration = 19

B. Show the pipeline timing assuming **full forwarding is enabled** and register file supports **same-cycle read/write.** Show the forwarded values and indicating stalls; i.e, by circling or drawing clouds as well as labeling the instructions. Hint: recall that stalling does not use NOP instructions.

How many **clock cycles** are required for **one iteration** now?

Instruction	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6	CC 7	CC 8	CC 9	CC 10	CC 11	CC 12	CC 13	CC 14
lw	IF1	ID1	EX1	MEM1	WB1									
sub		IF2	ID2	-	EX2	MEM2	WB2							
add			IF3	-	ID3	EX3	MEM3	WB3						
and					IF4	ID4	EX4	MEM4	WB4					
addi						IF5	ID5	EX5	MEM5	WB5				
beq							IF6	ID6	EX6	MEM6	WB6			
add								IF7	ID7	EX7	MEM7	WB7		

Stalling → CC 4: (sub & add instructions)

Forwarding:

MEM1 → EX2
EX2 → EX3
EX3 → EX4
EX4 → EX5
EX5 → ID6

Cycles for 1 iteration = 11

Class Activity- Pipelined RISC-V

Using a diagram similar to Figure 7.59, show the flush needed to execute the instructions from Q3 on the pipelined RISC-V processor.

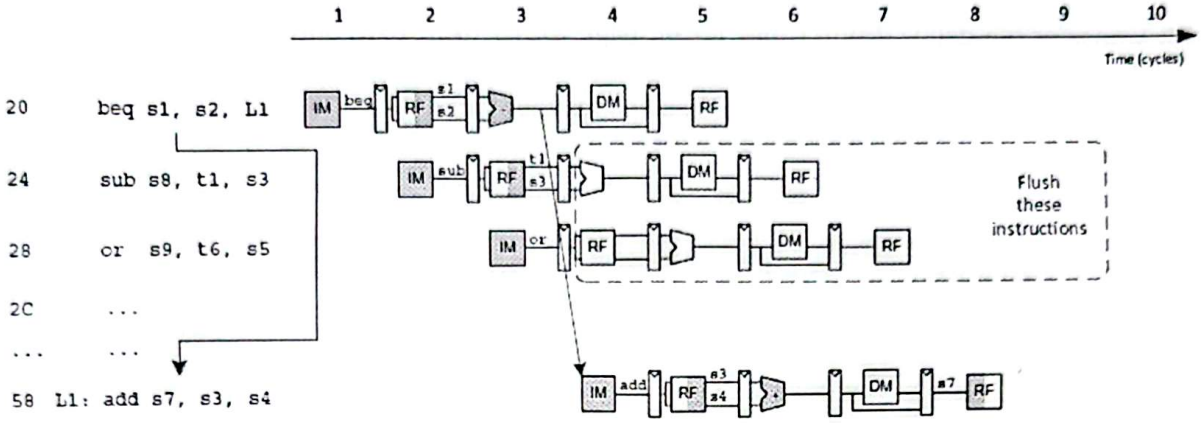
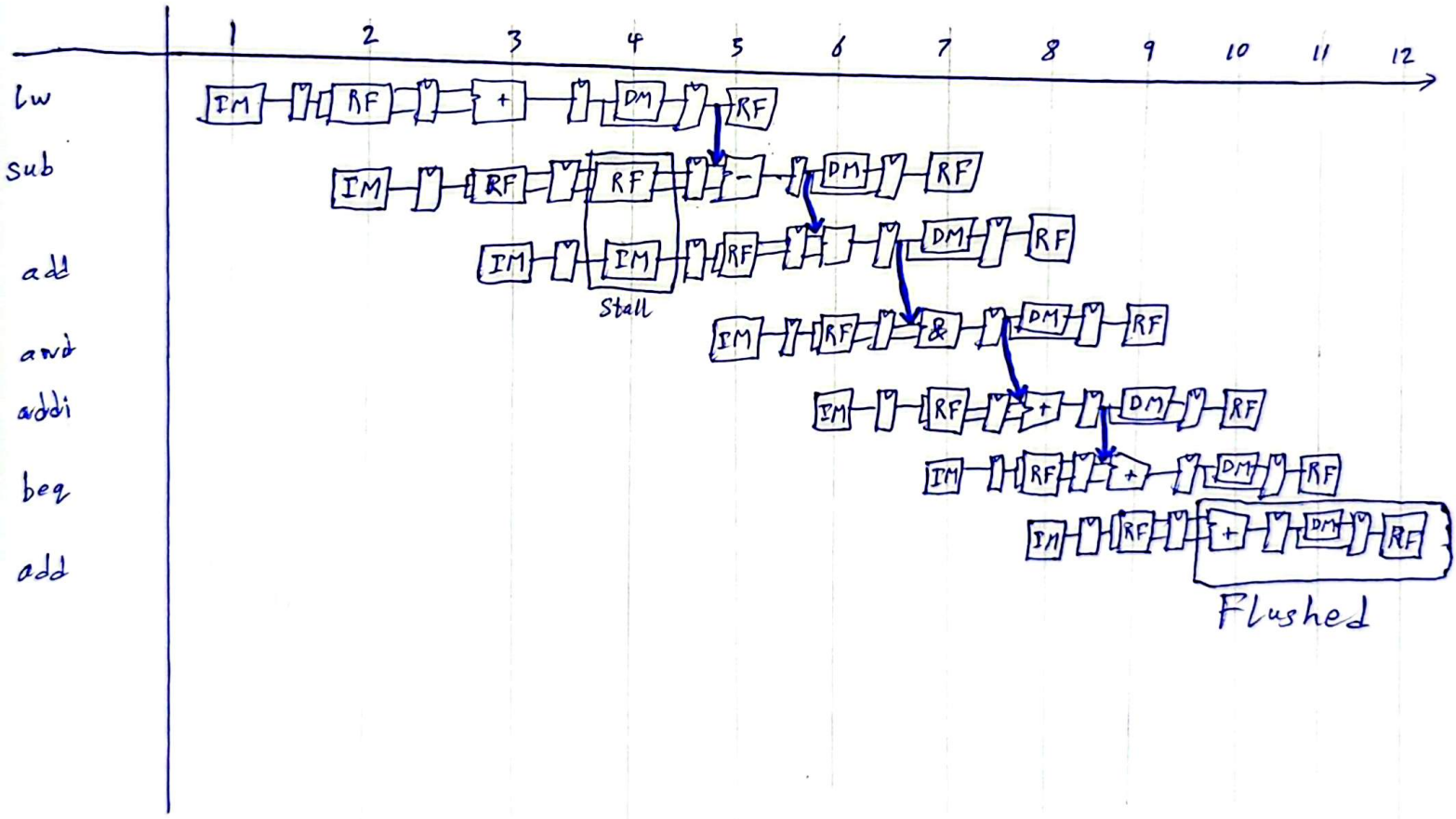


Figure 7.59 Abstract pipeline diagram illustrating flushing when a branch is taken



Class Activity- Pipelined RISC-V

Q4: Consider the delays from Table 7.7 on page 415(book). Now, suppose that the ALU were 20% faster. Would the cycle time of the pipelined RISC-V processor change? What if the ALU were 20% slower? Explain your answers and show your work.

Table 7.7 Delay of circuit elements

Element	Parameter	Delay (ps)
Register clk-to-Q	t_{pcq}	40
Register setup	t_{setup}	50
Multiplexer	t_{mux}	30
AND-OR gate	t_{AND-OR}	20
ALU	t_{ALU}	120
Decoder (control unit)	t_{dec}	25
Extend unit	t_{ext}	35
Memory read	t_{mem}	200
Register file read	t_{RFread}	100
Register file setup	$t_{RFsetup}$	60

Delay times:

Fetch: $t_{pcq} + t_{mem} + t_{setup} = 290$ ps

Decode: $t_{pcq} + t_{RFread} + t_{setup} = 190$ ps

Execute: $t_{pcq} + t_{mux} + t_{ALU} + t_{setup} = 240$ ps

Memory: $t_{pcq} + t_{mem} + t_{setup} = 290$ ps

Writeback: $t_{pcq} + t_{mux} + t_{RFsetup} = 130$ ps

Max delay = 290 ps

Scenerio-A:

ALU 20% faster

⇒ New delay = 120 ps × 0.8

New Execute delay = 40 + 30 + 96 + 50 = 216 ps

⇒ Cycle time will not change bcz Fetch & Mem take 290 ps so the cycle will still have to be slower for them

Scenerio-B:

ALU 20% slower

New ALU delay = 120 ps × 1.2 = 144 ps

New Execute delay = 40 + 30 + 144 + 50 = 264

Cycle time will not change due to same prev reason